UNIVERSITY OF CALCUTTA

Federated Learning

M.Sc. Semester III

Paper Code: CSMP-306

Session: 2025-26

Submitted By

Debottam Kar

CU Roll: C91/CSC/241011

Reg. No: 313-1111-0359-21

Contents

1 Introduction: What is Federated Learning? 3

2 Traditional Machine Learning vs Federated Learning 3
2.1 Key Characteristics . . . 4
2.2 Key Characteristics . . . 5

3 Federated Learning in Depth 5
3.1 Core Idea . . . 5
3.2 System Architecture . . . 6
3.2.1 Participants . . . 6
3.2.2 Training Workflow . . . 6
3.3 Key Design Considerations . . . 7
3.4 Training Variants . . . 8
3.4.1 1. Synchronous Federated Learning . . . 8
3.4.2 2. Asynchronous Federated Learning . . . 9
3.4.3 3. Hybrid Approaches (FedBuff) . . . 10

4 Types of Federated Learning 11
4.1 Horizontal Federated Learning (HFL) . . . 11
4.2 Vertical Federated Learning (VFL) . . . 12
4.3 Federated Transfer Learning (FTL) . . . 13
4.4 Summary Comparison of FL Types . . . 14

5 Privacy Techniques 14

6 Optimization Challenges 15
6.1 Non-IID Data . . . 15
6.2 Unbalanced Data . . . 15
6.3 System Heterogeneity . . . 15
6.4 Communication Bottleneck . . . 15

7 Advanced Techniques 15
7.1 Gradient Compression . . . 15
7.2 Personalization . . . 15
7.3 Meta Learning in FL . . . 15
7.4 Federated Averaging Variants . . . 15

8 Real-World Applications 16
8.1 Healthcare . . . 16
8.2 Finance . . . 16
8.3 Mobile Devices . . . 16
8.4 Real-Life Application of Federated Learning: Google Keyboard . . . 16

9 Security Risks 16

10 Advantages of Federated Learning 17

11 Disadvantages of Federated Learning **17**

12 References **17**

1 Introduction: What is Federated Learning?

Healthcare has evolved significantly with the integration of technology, enabling processes like patient registration, remote monitoring, and self-care applications. The rise of the Internet of Medical Things (IoMT) has dramatically increased the volume of digital health data, and Artificial Intelligence (AI) is now used to analyse this data for applications like disease prognosis and personalized care.

However, this growth brings serious challenges around data privacy and security. Healthcare organizations handle vast amounts of sensitive patient information — medical records, diagnoses, treatment histories — and protecting this data from breaches has become a critical concern.

Deep learning (DL), a powerful branch of AI, plays a dual role in this landscape: it helps to analyse patient data for disease diagnosis and treatment planning, while also strengthening cybersecurity defence against threats and fraud in healthcare systems.

To address the tension between data utility and privacy, Federated Learning (FL) has emerged as a promising solution. Rather than centralizing raw patient data, FL trains machine learning models directly on decentralized sources — individual hospitals, clinics, or patient devices — and shares only model updates with a central server. This keeps sensitive data localized and secure. FL systems are built around three core components: client devices, a federated server, and a communication protocol.

In essence, the text frames federated learning as a natural response to the growing need for privacy-preserving AI in an increasingly data-driven healthcare environment.

Federated learning is a machine learning approach where multiple devices or organizations collaboratively train a shared model without sharing their raw data.

Instead of sending data to a central server, each participant:

1. Downloads the current global model.
2. Trains it locally on their own data.
3. Sends only the updated model parameters (not the data) back to a central server.
4. The server aggregates all updates to improve the global model.
5. The process repeats.

2 Traditional Machine Learning vs Federated Learning

In traditional approaches, all data is collected in one place and used to train a model.

- Data from multiple sources is sent to a central server
- The model is trained on this centralized dataset
- After training, the model is deployed

Example: A company collects all user data (images, text, transactions) on its servers and trains a model there.

2.1 Key Characteristics

- Centralized data storage
- Easier to manage and train
- Requires moving data to one location
- Higher risk of data breaches
- Huge Bandwidth usage (Huge data transfer)

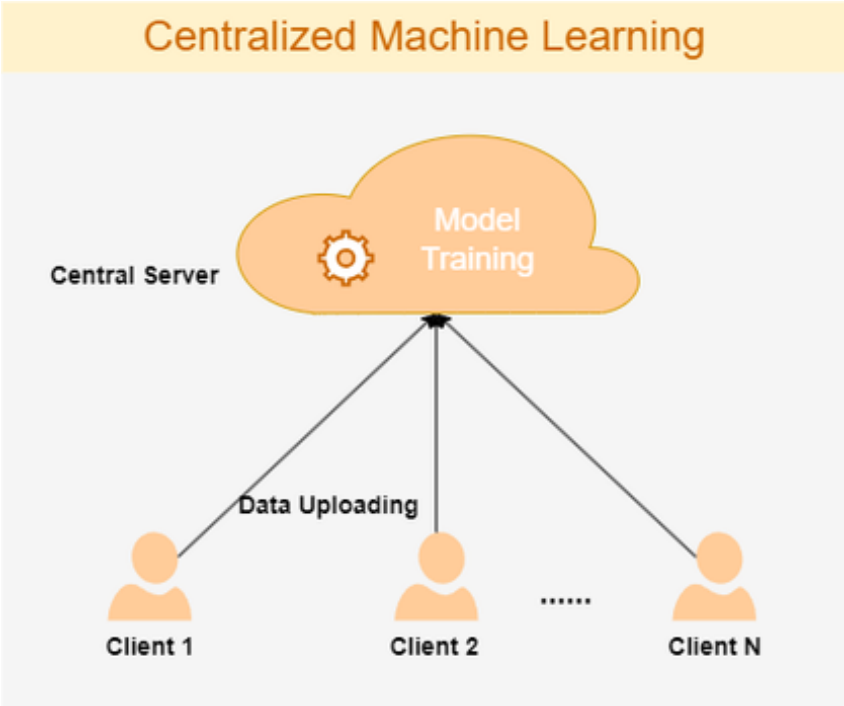

Figure 1: Centralized Machine Learning

Federated learning trains a model across multiple devices without collecting raw data.

- Data stays on local devices (phones, hospitals, banks, etc.)
- Each device trains the model locally
- Only model updates (not data) are shared with a central server
- The server aggregates updates to improve the global model

Example: Your phone improves a keyboard prediction model using your typing data—without sending your messages anywhere.

2.2 Key Characteristics

- Decentralized data
- Stronger privacy protection
- Data never leaves the device
- More complex training process

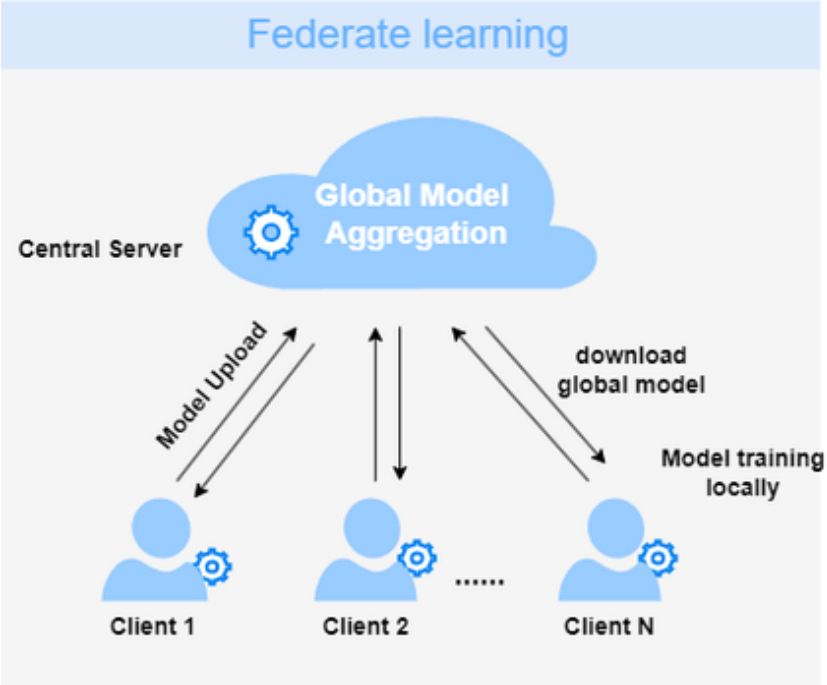

Figure 2: Federated Learning Architecture

3 Federated Learning in Depth

3.1 Core Idea

Instead of bringing data to the model, Federated Learning brings the model to the data.

- Each client (device/organization) keeps its local dataset
- A shared global model is collaboratively trained
- Only model updates (gradients or weights) are exchanged

3.2 System Architecture

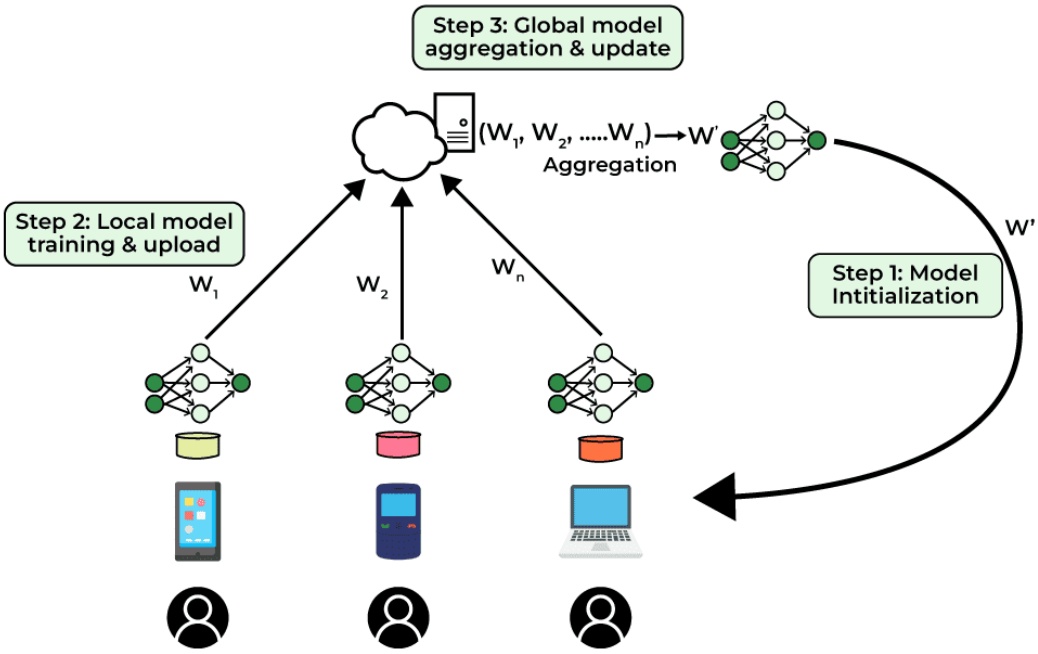

Figure 3: Federated Learning System Architecture

3.2.1 Participants

Server (Coordinator):
The server maintains the global model and is responsible for orchestrating the entire training process. It selects which clients participate in each round, distributes the current model to them, and aggregates their updates to produce an improved global model. It never sees raw data — only model updates.

Clients (Edge Devices)
Clients are the data-holding participants: smartphones, hospitals, banks, IoT sensors, etc. Each client trains the shared model on its own local data and sends only the resulting weight updates back to the server. Raw data never leaves the device.

3.2.2 Training Workflow

Step 1: Server initializes the global model
The server randomly initializes a global model $\mathcal{W}_0$ (using Gaussian, Xavier, or He initialization). This defines the shared architecture for all participants. A round counter t is set to 0.

Step 2: Server selects a subset of clients
At the start of each round t, the server selects a random subset of $C \cdot |S|$ clients (where C is the participation fraction, typically 10–30%). Selection can be uniform or weighted by data volume. Non-selected clients remain idle for that round.

Step 3: Server broadcasts the current model
The current global weights $\mathcal{W}_t$ are sent to every selected client. In practice, weights are compressed or quantized before transmission to reduce communication cost.

Step 4: Clients train locally on private data
Each selected client runs E epochs of mini-batch gradient descent on its local dataset $\mathcal{D}_k$, starting from the received $\mathcal{W}_t$. The local gradient is computed as:

$$\nabla L(\mathcal{W}; x_i, y_i) \quad \text{— evaluated only on local samples}$$

The resulting local model becomes:

$$\mathcal{W}_k^{(t+1)} = \mathcal{W}_t - \eta \cdot \nabla L_k(\mathcal{W}_t)$$

Only the weight delta $\Delta\mathcal{W}_k$ is prepared for upload. No raw data is transmitted.

Step 5: Clients upload updates; server aggregates
Each selected client sends its $\Delta\mathcal{W}_k$ (or full $\mathcal{W}_k^{(t+1)}$) to the server. The server applies the FedAvg algorithm:

$$\mathcal{W}_{t+1} = \sum \frac{n_k}{n} \cdot \mathcal{W}_k^{(t+1)}$$

This is a weighted average across clients, where n_k is the number of local samples on client k and n is the total samples across all participating clients. The new global model implicitly reflects learning from all clients' private data without the server ever seeing it.

The round counter increments to $t + 1$ and the cycle repeats until convergence.

3.3 Key Design Considerations

- **FedAvg weighting:** clients with more data exert proportionally greater influence on the global model. A hospital with 10,000 records contributes more than a phone with 500.
- **Communication efficiency:** Gradient sparsification, quantization (8-bit vs. 32-bit floats), and sketching can reduce communication cost by 10–100×.
- **Client drift:** More local epochs (higher E) reduces the number of communication rounds needed, but causes each device's model to overfit its local distribution, degrading aggregation quality.
- **Stragglers:** Synchronous aggregation waits for the slowest client; asynchronous aggregation risks incorporating stale gradients. Both are active research problems.
- **Privacy amplification:** Gradient updates can still leak training data via model inversion attacks. Production deployments layer on differential privacy (calibrated Gaussian noise added to gradients) and secure aggregation (cryptographic protocols ensuring the server sees only the sum of updates, never individual ones).
- **Non-IID data:** FedAvg converges well only when data is identically distributed across clients, which rarely holds in practice. Alternatives like FedProx (adds a proximal regularization term) and SCAFFOLD (corrects for client drift with control variates) address this limitation.

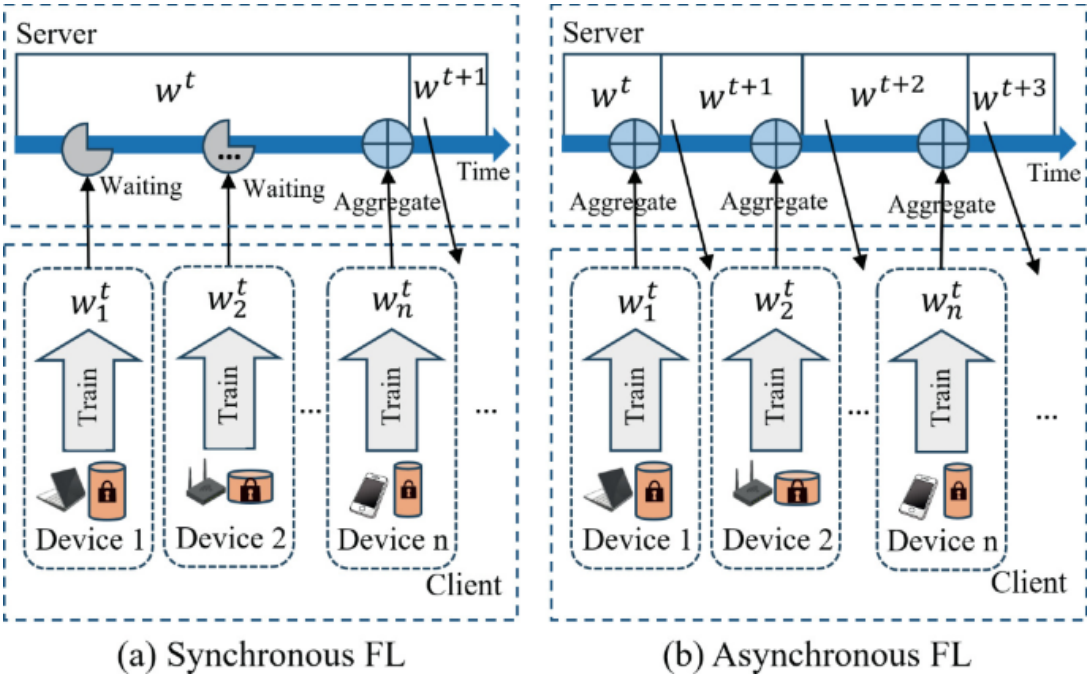

Figure 4: Synchronous vs Asynchronous Federated Learning

3.4 Training Variants

3.4.1 1. Synchronous Federated Learning

Core Concept: Synchronous FL operates on a *"wait-for-all"* principle. In each communication round, the server selects a set of clients, distributes the current global model, and then waits to receive updates from **every** selected client (or a predefined quorum) before aggregating them and proceeding to the next round.

Mechanics:

1. **Round t Begins:** The server broadcasts the global model w_t to a cohort of K clients.
2. **Local Training:** Each client trains locally for E epochs.
3. **The Barrier:** The server halts and waits. It does not proceed until it has received updates from all K clients (or a strict majority).
4. **Aggregation:** The server aggregates all received updates (via Federated Averaging) to produce the new global model w_{t+1}.
5. **Proceed:** The next round begins.

Advantages:

- **Mathematical Stability:** Convergence guarantees are well-established for synchronous methods.
- **Model Coherence:** Every update in round t starts from the exact same global model w_t, preventing gradient staleness.

- **Simpler Aggregation:** Standard algorithms like FedAvg are designed for synchronous settings.

Disadvantages:

- **The Straggler Problem:** The entire round is as slow as the slowest client.
- **Inefficiency:** Powerful, fast clients must wait idle for slow clients.
- **Dropout Sensitivity:** If a client drops out, the server must wait for a timeout or discard that client's work.

Variations (Mitigating Stragglers):

- **Quorum-based Sync:** Server waits for a minimum percentage (e.g., 80%) of clients.
- **Federated Averaging with Timeouts:** Server sets a hard deadline; late clients are dropped.

Real-World Example: Google's Gboard initial implementation used synchronous rounds, selecting hundreds of thousands of phones and waiting for a specific number (e.g., 200) to return updates.

3.4.2 2. Asynchronous Federated Learning

Core Concept: Asynchronous FL operates on an *"update as you go"* principle. There are no fixed rounds. The server maintains a single global model, and whenever *any* client finishes local training and sends its update, the server immediately applies that update to the global model.

Mechanics:

1. **Continuous State:** The server holds a global model w. Clients can request it at any time.
2. **Independent Pull:** Client A downloads the current global model (version w_t), trains locally, and sends the update back.
3. **Immediate Push:** The server receives Client A's update and immediately computes:
$$w_{new} = w_{old} + \eta \cdot \Delta w_{client}$$
4. **Concurrency:** While the server applies Client A's update, Client B may download the new model w_{new}, and Client C may upload an update based on an older version.

Advantages:

- **No Stragglers:** Fast clients are never blocked by slow clients.
- **High Throughput:** Server can ingest thousands of updates per minute.
- **Fault Tolerance:** Client failures don't hold up the system.

Disadvantages:

- **Gradient Staleness:** Updates based on old model versions can push the model in the wrong direction.
- **Convergence Instability:** Asynchronous updates can cause oscillation or divergence.
- **Fairness Issues:** Fast clients can dominate the updates.

Mitigations (Handling Staleness):

- **Staleness-Aware Weighting:** Older updates are given lower weights:

$$w_{new} = w_{old} + \eta(\tau) \cdot \Delta w$$

  where τ is the staleness (higher τ = lower η).

- **Hedge Updates:** Algorithms like Hogwild! or FedAsync adjust aggregation based on staleness.

Real-World Example: Cross-silo FL with hospitals across different time zones. Hospitals submit updates whenever they have spare compute capacity, and the server updates the global model continuously without waiting for coordination.

3.4.3 3. Hybrid Approaches (FedBuff)

Modern FL often uses a hybrid model known as **Buffered Asynchronous Aggregation (FedBuff)**:

- Clients train asynchronously (no barriers)
- Server deposits incoming updates into a **buffer**
- Once the buffer reaches a target size (e.g., 100 updates), the server performs a **synchronous aggregation**

This combines the throughput of asynchronous with the stability of synchronous methods.

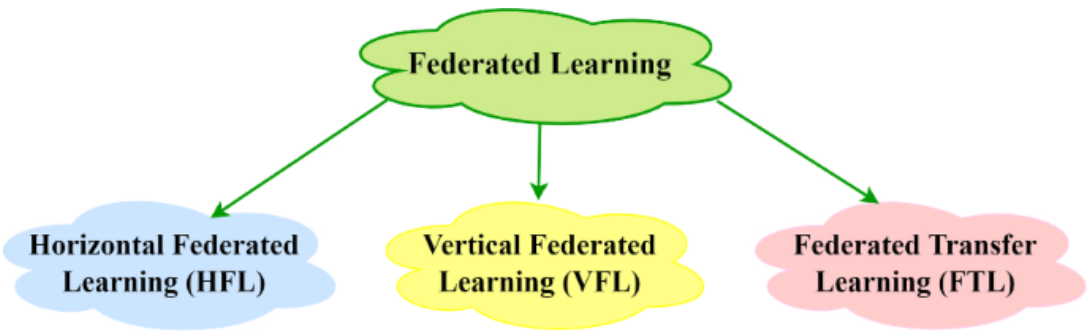

Figure 5: Types of Federated Learning

4 Types of Federated Learning

Federated Learning is categorized based on how data is distributed across participants in terms of **sample space** (users) and **feature space** (data attributes).

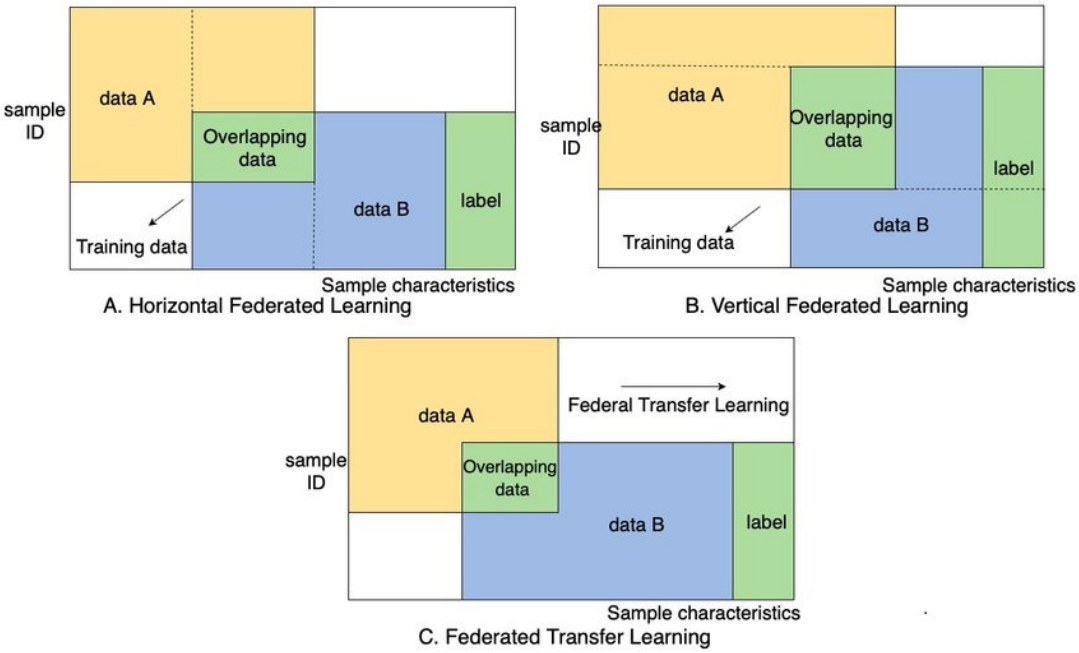

Figure 6: Horizontal, Vertical, and Transfer Federated Learning

4.1 Horizontal Federated Learning (HFL)

Also known as: Sample-Based Federated Learning or Homogeneous FL.

Core Concept: In HFL, datasets share the **same feature space** but consist of **different samples (users)**. The data columns (features) align perfectly, but rows (samples) do not overlap.

Example: Two hospitals in different cities. Both have the same types of data (patient age, blood pressure, medical history) but on completely different sets of patients.

Detailed Mechanics:

1. **Initialization:** A central server initializes a global model and sends it to all participating clients.

2. **Local Training:** Each client trains the model using its own local data. Feature spaces are identical, so model architecture is the same.

3. **Aggregation:** Clients send only model updates back to the server—never raw data.

4. **Fusion:** Server aggregates updates using **Federated Averaging (FedAvg)**, averaging weights proportionally to dataset sizes.

5. **Iteration:** The updated global model is sent back down.

Key Characteristics:

- **Data Distribution:** Partitioned by rows (samples)
- **Architecture:** Typically requires a central coordinator
- **Challenges:**
  - **Statistical Heterogeneity (Non-IID):** Data across users is rarely identically distributed
  - **Communication Efficiency:** Thousands of clients require compression algorithms
  - **Client Drift:** Local models can drift too far from the global model

Real-World Example: Mobile Keyboard Prediction (Google Gboard)

- Each user's typing history stays on their device
- Features (last three words typed, app context) are identical across all phones
- Goal: Train a next-word prediction model without uploading private typing history

4.2 Vertical Federated Learning (VFL)

Also known as: Feature-Based Federated Learning or Heterogeneous FL.

Core Concept: In VFL, datasets share the **same sample space** (same users or entities) but have **different feature spaces**. Organizations hold different attributes about the same individuals and collaborate without exposing proprietary data.

Example: A bank and an e-commerce company have the same customers but different data—bank has financial history, e-commerce has purchase behavior.

Detailed Mechanics:

1. **Entity Alignment (Private Set Intersection - PSI):** Cryptographic protocols identify common users without revealing users they don't share.
2. **Feature Partitioning:** Model is split vertically. Usually one party holds labels (target variable) while others hold features.
3. **Encrypted Computation:** Parties exchange encrypted intermediate results using **Homomorphic Encryption** or **Secret Sharing**.
4. **Cooperative Gradient Calculation:** Gradients are calculated using additive encryption—parties combine encrypted gradients without seeing raw data.

Key Characteristics:

- **Data Distribution:** Partitioned by columns (features)
- **Architecture:** Often requires a third-party "arbiter" for key management
- **Challenges:**
  - **Sample Alignment:** Finding user intersections without leaking data is computationally heavy

  - **Label Skew:** Party holding labels has leverage and must be prevented from inferring others' features
  - **Complexity:** Requires heavy cryptographic primitives with significant overhead

Real-World Example: Bank + E-commerce Collaboration

- Bank: User IDs, income, existing loans, label (Defaulted: Yes/No)
- E-commerce: User IDs (overlapping), average cart value, category preferences, return rate
- Goal: Use behavioral data to improve credit risk prediction without sharing proprietary data

4.3 Federated Transfer Learning (FTL)

Also known as: Heterogeneous Federated Learning without Overlap.

Core Concept: FTL addresses the most challenging scenario: participants have **different users** (little to no sample overlap) AND **different features** (little to no feature overlap). It leverages **Transfer Learning** techniques to bridge the gap between heterogeneous datasets.

Example: A social media platform (Twitter) and a music streaming service (Spotify) with no common users and completely different features (text vs. audio).

Detailed Mechanics:

1. **Feature Extraction:** Each party uses local data to train a base model extracting high-level "knowledge" or embeddings.
2. **Mapping:** Techniques like **Adversarial Networks** (GANs) or **Matrix Factorization** find a common intermediate representation (shared latent space).
3. **Knowledge Distillation:** Instead of averaging incompatible gradients, FTL uses distillation—a "teacher" model learns aggregated knowledge from "student" models by comparing outputs (logits).
4. **Limited Intersection:** Some implementations use a tiny, encrypted overlap of entities as "anchor points" to align latent space.

Key Characteristics:

- **Data Distribution:** Disjoint rows (users) and disjoint columns (features)
- **Architecture:** Highly flexible; clients can have completely different architectures
- **Challenges:**
  - **Semantic Alignment:** Mapping between disparate modalities (e.g., medical images to financial text) without labeled paired data
  - **Complexity:** Most computationally expensive form of FL
  - **Performance Ceiling:** Transfer effectiveness is limited; used when other forms are impossible

Real-World Example: Cross-Domain Recommendation

- Twitter: User A (text data, likes, follower networks)
- Spotify: User B (audio features, listening history, playlists)
- Goal: Find latent embeddings linking "users who tweet about indie rock" to "users who listen to indie playlists" to improve recommendations for both platforms

4.4 Summary Comparison of FL Types

Feature	**Horizontal FL (HFL)**	**Vertical FL (VFL)**	**Federated Transfer Learning (FTL)**
Data Overlap	High Row Overlap (Samples)	High Column Overlap (Features)	Low/Minimal Overlap (Both)
Data Distribution	Same features, Different users	Same users, Different features	Different users, Different features
Key Challenge	Non-IID data, Client Drift	Entity Alignment (PSI), Label Security	Semantic Alignment, Latent Space Mapping
Primary Technique	Federated Averaging (FedAvg)	Homomorphic Encryption, Secure Multi-Party Computation	Transfer Learning, Knowledge Distillation
Use Case	Mobile keyboards, IoT devices	Cross-sector collaboration (Bank + Insurance)	Cross-domain collaboration (Social Media + E-commerce)
Model Architecture	Identical models across clients	Split-Network (vertical split)	Independent models + Bridge network

Table 1: Comparison of Horizontal, Vertical, and Federated Transfer Learning

5 Privacy Techniques

Federated learning is privacy-preserving, but not perfectly private. It often combines:

1. **Differential Privacy:** Adds noise to updates to prevent reconstruction of data.
2. **Secure Aggregation:** Server only sees the **sum of updates**, not individual ones.
3. **Homomorphic Encryption:** Allows computation on encrypted data.

6 Optimization Challenges

6.1 Non-IID Data

Data across clients is **not identically distributed**.

- Example: Different typing styles, different medical records
- Causes instability in training

6.2 Unbalanced Data

Some clients have more data than others.

6.3 System Heterogeneity

Devices vary in:

- CPU power
- Battery
- Connectivity

6.4 Communication Bottleneck

Sending model updates is expensive, especially for large models.

7 Advanced Techniques

7.1 Gradient Compression

Reduces communication size via:

- Quantization
- Sparsification

7.2 Personalization

- Global model + local fine-tuning
- Each client adapts model to its data

7.3 Meta Learning in FL

Learn a model that can quickly adapt to each client.

7.4 Federated Averaging Variants

- FedProx: handles non-IID data better
- Scaffold: reduces client drift

8 Real-World Applications

8.1 Healthcare

Hospitals collaboratively train models without sharing patient data.

8.2 Finance

Banks detect fraud without exposing sensitive customer data.

8.3 Mobile Devices

- Predictive keyboards
- Voice recognition
- Next-word prediction

8.4 Real-Life Application of Federated Learning: Google Keyboard

- **Data Collection:** Google Keyboard (Gboard) collects data, like the names of restaurants you search for, but keeps it on your phone. Your personal data never leaves your device.
- **Federated Learning in Action:** The model is trained directly on your phone using the data (like your search history). No need to send your data to a central server, keeping it private.
- **Model Updates:** After training locally, the updates to the model are sent to a central server in an encrypted form. The server combines updates from different devices to improve the model.
- **Better Suggestions:** As more updates are gathered, the model gets better at suggesting things, like more accurate restaurant names or typing predictions based on your habits.
- **Privacy and Efficiency:** Your data stays private, and only model updates are shared. This helps save bandwidth and makes everything run more efficiently.

9 Security Risks

Even though data isn't shared, attacks are still possible:

1. **Gradient Leakage:** Attackers may reconstruct data from gradients.
2. **Poisoning Attacks:** Malicious clients send fake updates to corrupt the model.
3. **Inference Attacks:** Infer whether a data point was part of training.

10 Advantages of Federated Learning

- **Reduced Power Usage:** Reduced data size means reduced computation time and hence less power usage.
- **Guarantees Privacy:** Data stays on the device, maintaining privacy without loss of training.
- **No Device Performance Impact:** Training is done only when the device is in idle or charging state, hence no impact on performance.
- **Scalability:** Scales with big, distributed datasets across multiple devices.
- **Better Model Performance:** Uses heterogeneous data from various devices to improve model accuracy.
- **Real-time Updates:** Allows real-time model updates on each device.

11 Disadvantages of Federated Learning

- **Network Latency:** The communication between the devices and the central server can be a bottleneck and may add latency to the training process.
- **Heterogeneous devices:** The devices can be heterogeneous in terms of hardware and software, which can make it difficult to ensure the compatibility and consistency of the models.
- **Data Quality:** The quality of data can vary across the devices, which can lead to poor model performance.

12 References

1. https://cloud.google.com/discover/what-is-federated-learning
2. https://www.researchgate.net/figure/Classification-of-federated-learning-in-10-a fig1_378809686
3. https://onlinelibrary.wiley.com/doi/10.1155/2021/8261663
4. https://www.youtube.com/watch?v=89BGjQYA0uE
5. https://www.geeksforgeeks.org/machine-learning/collaborative-learning-federated-
6. https://media.springernature.com/lw685/springer-static/image/chp%3A10.1007%2F978-981-95-3543-9_21/MediaObjects/654870_1_En_21_Fig1_HTML.png
7. https://media.springernature.com/lw685/springer-static/image/art%3A10.1007%2Fs10115-024-02257-6/MediaObjects/10115_2024_2257_Fig8_HTML.png